Лабораторная работа №3

Автоматические проверки, миграции и резервное копирование

Задание

1. Создать единый сценарий первоначальной настройки проекта (см. вкладку Общие требования).

```
В терминале VS Code
.\Makefile.ps1 setup
Эта команда:
```

- Использует существующую .venv, если она уже есть.
- Устанавливает зависимости из requirements.txt.
- Не перезаписывает твой .env, если он существует.
- Создаёт папку backups, если её ещё нет.
- Не изменяет базу данных, не запускает миграции и не меняет пароли.

2. Зафиксировать версии основных зависимостей.

В файле requirements.txt зафиксированы все версии основных зависимостей

3. Добавить команды установки, запуска, проверки, миграции, резервного копирования и восстановления.

```
.\Makefile.ps1 setup установка
.\Makefile.ps1 run запуск
.\Makefile.ps1 test запуск тестов на питоне
.\Makefile.ps1 migrate (миграция) с помощью инструмента алембика,
восстанавливаем старую структуру бд, которую прежде сохранили.
.\Makefile.ps1 backup сделать резервную копию бд
Get-ChildItem .\backups посмотреть место хранения копий и хранящиеся копии
.\Makefile.ps1 restore начать процесс восстановления копии
```

4. Написать unit-тесты для основной логики приложения и интеграционные тесты работы с базой данных (тестирование должно покрывать не менее 40% от функционала, описанного в ТЗ).

```
.\.venv\Scripts\python.exe -m pytest --cov=main --cov-report=term-missing мои
юнит тесты (tests/test_main.py — файл с тестами.; tests/conftest.py —
настройка тестового окружения.; beetles_test_db — отдельная тестовая база
PostgreSQL.; pytest-cov — инструмент измерения покрытия кода.)
```

5. Добавить проверки ошибочных данных, граничных случаев и обязательных правил работы приложения (см. мутационное тестирование).

.\Makefile.ps1 test запускаем исполняющий файл с тестами на питоне. проверки ошибочных данных, граничных случаев и мутационное тестирование. (пример: if expedition.end_time < expedition.start_time: в файле main.py это критически важное условие в котором можно поставить <= и тест поймает ошибку в коде. измененная программа это и есть мутант)

6. Организовать отдельную тестовую конфигурацию и изоляцию тестовых данных.

\tests\conftest.py код отдельной тестовой конфигурации и изоляции тестовых данных с помощью использования данных из .env.test. (в коде прописано использование данных из этого файла)

7. Добавить проверку форматирования (линтер), статический анализ кода на безопасность (SAST) , отчеты тестов и покрытия.

.\.venv\Scripts\python.exe -m ruff format --check main.py database.py tests подтверждает, что форматирование всех проверяемых Python-файлов соответствует настройкам Ruff. (ruff это инструмент на языке питон для проверки качества кода и форматирования)
.\.venv\Scripts\python.exe -m ruff check main.py database.py tests проверяет основной код приложения (main.py, database.py) и тесты (tests) на ошибки, нарушения правил программирования и потенциально проблемные конструкции
.\.venv\Scripts\python.exe -m bandit -r main.py database.py выполняет статический анализ безопасности (SAST), проверяя Python-код на потенциальные уязвимости без запуска приложения
.\.venv\Scripts\python.exe -m bandit -r main.py database.py -f json -o reports\bandit-report.json проверяет безопасность кода и сохраняет результаты проверки в JSON-файл reports\bandit-report.json
.\.venv\Scripts\python.exe -m pytest tests -q --junitxml=reports\pytest-results.xml --cov=main --cov=database --cov-report=term-missing --cov-report=html:reports\coverage-html --cov-report=xml:reports\coverage.xml запускает автоматические тесты, проверяет их успешность, измеряет процент покрытия кода тестами и создаёт отчёты в форматах XML и HTML
Start-Process .\reports\coverage-html\index.html открывает HTML-отчёт покрытия кода в браузере, где можно посмотреть процент покрытия и строки, не охваченные тестами.

Get-Content .\ruff.toml выводит полный конфигурационный файл Ruff, где указаны действующие настройки проверки.

8. Подключить средство миграций и создать не менее трех последовательных изменений схемы БД.

Чтобы провести миграцию, нужно сделать новую ревизию, сделать в ней какое то изменение (скопировать пример изменения из другой ревизии), затем применить конкретно эту ревизию

python -m alembic revision -m "название" Создаёт файл новой миграции
.\.venv\Scripts\python.exe -m alembic history --verbose или python -m alembic history Показывает историю миграций
python -m alembic current Показывает текущую версию БД
python -m alembic upgrade head привести версию бд к последней из возможных

```
ревизий
python -m alembic downgrade -1 Откатывает одну миграцию
.\.venv\Scripts\python.exe -m alembic upgrade 4c5d922cd5b3 применить
конкретную версию миграции
SELECT * FROM alembic_version; посмотреть версию бд алембика в моей консоли
(psql)
```

SQL SQL SQL

\l все дб

\c конект к дб

\d отобразить все таблицы

\d имятаблицы - отобразить содержимое конкретной таблицы

9. Проверить создание чистой БД и обновление БД, уже содержащей данные.

```
CREATE DATABASE beetles_migration_lab_db; создаем новую пустую бд для теста
миграции на новую бд
\i 'C:/Users/Admin/Desktop/code/beetles_app/beetles_schema.sql' скопировать
схему моей бд (вводится в (psql)
\dt public.* посмотреть что схема бд правильная, таблицы создались все

Для применения миграций на новую бд на которую мы скопировали схему, нужно
прописать имя этой бд в файле .env.migration_clean
+
migrations_clean\env.py нужно поменять имя бд


python -m alembic -c alembic_clean.ini current посмотреть какая версия
миграции последняя в этой области миграции

python -m alembic -c alembic_clean.ini upgrade head чтобы применить все
версии наших миграций в этой области миграции
```

10. Добавить новую функцию приложения, требующую изменения схемы.

```
pg_dump -U postgres -h localhost -p 5432 --schema-only -d ИМЯ_БД db1 -f
ИМЯ_СХЕМЫ_БД.sql
```

11. Реализовать команды создания резервной копии и восстановления.

```
pg_dump -U postgres -h localhost -p 5432 -F c -f
".\backups\ПУТЬ_И_НАЗВАНИЕ_НОВОГО_БЕКАПА.backup" ИМЯ_БД_КОТОРУЮ_БЕКАПИМ

Get-Item .\backups\ПУТЬ_И_НАЗВАНИЕ_НОВОГО_БЕКАПА.backup | Select-Object Name,
Length проверка что бекап создался

ПЕРЕД ВОССТАНОВЛЕНИЕМ БД ИЗ БЕКАПА, НУЖНО СОДЗАТЬ НОВУЮ ПУСТУЮ БД
createdb -U postgres -h localhost -p 5432 beetles_migration_lab_db

ВОССТАНАВЛИВАЕМ БД ИЗ БЕКАПА ВМЕСТЕ С ДАННЫМИ И СТРУКТУРОЙ
pg_restore -U postgres -h localhost -p 5432 -d beetles_migration_lab_db --
exit-on-error --no-owner --no-privileges -v ".\backups\beetles_lab.backup"
```

```
ПОСЛЕ ВОССТАНОВЛЕНИЯ МОЖНО ПОСМОТРЕТЬ ЧТО ТАБЛИЦА ЗАПОЛНЕННА ДАННЫМИ ИЗ БЕКАПА
psql -U postgres -h localhost -p 5432 -d beetles_migration_lab_db -c "SELECT * FROM public.places;"
```

12. Выполнить восстановление данных после намеренного удаления или изменения.

```
dropdb -U postgres -h localhost -p 5432 --if-exists --interactive ИМЯ_БД_КОТОРУЮ_УДАЛЯЕМ полное удаление бд
```

13. Создать единую команду make verify, запускающую весь обязательный набор локальных проверок.

```
.\Makefile.ps1 verify автоматически запускает Ruff, Bandit и Pytest с измерением покрытия
Ruff ошибок в коде питона нет
Bandit Уязвимостей не обнаружено в коде на питоне
Pytest 29 из 29 тестов функционала
Coverage 94% общего покрытия
```

14. Зафиксировать запрет на пропуск тестов, снижение порогов и добавление исключений анализаторов без обоснования.

```
.\.venv\Scripts\python.exe -m quality_policy.check_quality_policy
```

Здесь используется библиотека ast — она разбирает Python-код и позволяет находить определённые конструкции.

Скрипт ищет:

- pytest.skip() — пропуск теста;
- pytest.skipif() — условный пропуск;
- pytest.xfail() — обозначение ожидаемого сбоя;
- декораторы @pytest.mark.skip, @pytest.mark.skipif, @pytest.mark.xfail.

в файле check_quality_policy.py в строке12 расписали 2 правила исключения сканирования, тк это ложно позитивные срабатывания

15. Провести одно функциональное изменение только через полный набор проверок make verify.

```
.\Makefile.ps1 verify сама коанда которая прогоняет все тесты

133 строка test_main.py

20 строка main.py
```

python -m ruff check . –fix исправить ошибки проверки ruff

Что именно мы сделали для пункта 15

1. Добавили новую функциональность: HTTP-эндпоинт GET /health, который возвращает {"status": "ok"}.
2. Написали тест: test_health_check(), проверяющий код ответа 200 и содержимое JSON.
3. Запустили verify: сначала Ruff обнаружил лишние пробелы и остановил проверку.
4. Исправили ошибку и повторили verify: все этапы успешно завершились.
5. До изменения было 29 тестов и 90,87% покрытия. После изменения — 30 тестов и 90,98% покрытия.

Типо функционал это проверить что сервер отвечает с помощью ответа апишки http://127.0.0.1:8000/health. Этим я проверяю работу именно FastAPI а не просто что мой код сайта работает

Проверка на защите

· преподаватель изменяет одно условие в основной логике, и тесты должны обнаружить изменение;

· проверка миграций на чистой и заполненной БД;

· создание резервной копии, повреждение данных и восстановление;

· объяснение назначения каждой команды в make verify.

Результат работы

Проект имеет единый локальный проверочный контур, которому подчиняется любое изменение.